

## COMMON COURSE ASSIGNMENT FORMAT

### A. Assignment Information

| Particular                | Details                                                                                                                                        |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Department:               | Computer Science and Engineering                                                                                                               |
| Programme:                | B.Tech. / UG Programme                                                                                                                         |
| Course Code & Course Name | CSA1704 & Artificial Intelligence                                                                                                              |
| Academic Year / Batch     | 2026 – 2027                                                                                                                                    |
| Faculty Name              | Dr. ANOOP M                                                                                                                                    |
| Assignment Title          | AI-Based Crop Disease Detection and Smart Agriculture Advisory System                                                                          |
| Date of Issue             | 01-09-2026                                                                                                                                     |
| Date of Submission        | 02-09-2026                                                                                                                                     |
| Maximum Marks             | 100                                                                                                                                            |
| Course Outcome(s) – CO    | CO3, CO4, CO5                                                                                                                                  |
| Bloom's Taxonomy Level    | L3 – Apply, L4 – Analyze, L5 – Evaluate, L6 – Create                                                                                           |
| SDG Mapping (SDG 1-17)    | SDG 2 – Zero Hunger; SDG 9 – Industry, Innovation and Infrastructure; SDG 12 – Responsible Consumption and Production; SDG 13 – Climate Action |

|                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Industry / Societal Relevance | The proposed AI-based system is highly relevant to the agricultural and agri-technology sectors. It supports early detection of crop diseases, helps reduce crop losses, enables timely decision-making and provides intelligent agricultural advisory support to farmers. The system can contribute to improved crop productivity, sustainable farming practices, reduced misuse of agricultural inputs, food security, and enhanced farmer livelihood. |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## B. Assignment Problem / Challenge

### AI-Based Crop Disease Detection and Smart Agriculture Advisory System

Agricultural crops are frequently affected by diseases caused by pathogens, environmental conditions, and other factors, which can reduce crop yield and affect farmers' income. Early and accurate identification of crop diseases is therefore essential for timely intervention and effective crop management.

The objective of this assignment is to design and develop an AI-based crop disease detection and smart agriculture advisory system that can identify crop diseases from available agricultural data, such as crop leaf images and relevant environmental or field parameters.

The proposed system shall:

1. Collect, preprocess, and analyze suitable crop disease datasets, such as leaf images and/or agricultural parameters.
2. Develop and evaluate an appropriate machine-learning or deep-learning model to classify healthy and diseased crops and identify the disease category where applicable.
3. Apply suitable AI techniques to analyze the detected disease and provide appropriate agricultural recommendations or advisory information.
4. Where applicable, incorporate environmental and field parameters such as temperature, humidity, soil conditions, or other available data to improve the advisory process.
5. Evaluate the performance of the proposed system using suitable metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

6. Analyze and interpret the results and justify the selection of the AI model and methodology based on prediction performance and computational efficiency.
7. Consider relevant technical, economic, environmental, societal, ethical, safety, and sustainability factors in Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge** the design and deployment of the proposed system.  
**or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

### C. Problem Statement

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.

### Problem / Case / Design Challenge:

- Consider requirements and constraints.

[Faculty to provide the detailed assignment problem here.]

- Develop or compare alternative solutions where appropriate.

Agricultural crops are frequently affected by diseases that can significantly reduce crop yield, quality,

- Use relevant modern engineering/computing/design tools.

and farmers' income. Early and accurate identification of crop diseases is essential for taking timely

### D. Requirements and Constraints

preventive and corrective measures. However, manual disease identification can be time-consuming, requires expert knowledge, and may not always be accessible to farmers.

Students shall identify or be provided with relevant requirements and constraints. The objective of this assignment is to **design and develop an AI-based Crop Disease Detection and Smart**

**Agriculture Advisory System** that can automatically identify crop diseases using suitable

Examples include:

agricultural data, such as crop leaf images and, where available, environmental or field parameters. • Functional requirements

The proposed system shall **preprocess and analyze the collected data, develop and evaluate a**

- Performance requirements

**suitable machine-learning/deep-learning model for disease classification, and provide** • Technical constraints

**appropriate agricultural advisory recommendations based on the detected disease.** The system • Cost, Time

should be validated using suitable performance measures such as accuracy, precision, recall, F1-score, • Resources

and confusion matrix.

- Safety

Students shall **analyze and interpret the results, justify the selection of the AI techniques and**

- Reliability

**model, and consider relevant factors such as computational efficiency, reliability, environmental**

- Sustainability

**sustainability, farmer usability, safety, and ethical considerations.**

- Environmental and Ethical considerations

- Standards/codes

#### **D. Requirements and Constraints**

- User requirements

- Manufacturability

1. The system shall accept suitable agricultural data, such as crop leaf images and, where available,

- Power/energy

environmental or field parameters.

- Security/privacy

2. The dataset shall be preprocessed to handle image resizing, normalization, noise, missing values, and other relevant data-quality issues.

**Only constraints relevant to the particular course/problem need to be considered.**

3. The system shall develop a suitable machine-learning/deep-learning model to identify healthy and diseased crops.
4. The model shall classify the type of crop disease, wherever the dataset supports disease-level classification.
5. The system shall provide appropriate agricultural advisory recommendations based on the detected disease.
6. The model shall be evaluated using suitable performance measures such as accuracy, precision, recall, F1-score, and confusion matrix.
7. The system shall provide appropriate visualizations and/or outputs to support interpretation of the prediction results.
8. Students shall document the methodology, model selection, results, limitations, and conclusions.

**Constraints:**

1. The solution shall use a reliable and relevant crop disease dataset.
2. Computational resources available to students shall be considered while selecting the AI/ML model.
3. The model should provide reasonable prediction accuracy and reliability for the selected crops

and disease categories.

4. The system should be designed to minimize false disease predictions, particularly where incorrect recommendations could affect crop health.
5. Any agricultural recommendations should be treated as decision-support information and should not be presented as a replacement for professional agricultural expertise.
6. Dataset limitations, such as limited crop varieties, disease classes, image quality, environmental conditions, or class imbalance, shall be identified and discussed.
7. Student solutions should consider data privacy, responsible AI usage, sustainability, affordability, and usability for farmers where applicable.
8. The proposed solution should be completed within the available time, software, hardware, and dataset constraints.

## **E. Student Work**

### **1. Problem Understanding and Formulation**

Agriculture is one of the most important sectors for food production and rural livelihoods. Crop diseases can reduce crop quality and yield when they are not identified early. Farmers may find it difficult to identify diseases only by observing symptoms, especially when different diseases have similar visual signs.

The proposed AI-Based Crop Disease Detection and Smart Agriculture Advisory System uses computer vision and deep learning to identify crop diseases from leaf images. The system uses MobileNetV2 with transfer learning and the PlantDoc dataset. After detecting the disease, the system provides a simple agriculture advisory such as possible precautions, monitoring suggestions, and recommended next steps.

The problem can therefore be formulated as an image classification problem where an input crop image is mapped to one of the disease classes learned during training.

### **2. Objectives**

The main objectives of the project are:

- Detect crop diseases from leaf images.
- Use MobileNetV2 transfer learning for image classification.

- Train and evaluate the model using the PlantDoc dataset.
- Provide a simple and understandable disease prediction.
- Generate smart agriculture advisory information based on the predicted disease.
- Support early identification and decision-making for farmers.
- Develop a system that can be extended into a web or mobile application.

### **3. Information and Data Available**

The system requires crop leaf images representing healthy and diseased conditions. The PlantDoc dataset is used as the main image dataset. The images are processed before being supplied to the deep learning model.

Important information used by the system includes:

- Crop/plant image.
- Disease or health class.
- Image dimensions and pixel values.
- Training and validation samples.
- Model prediction probabilities.
- Evaluation metrics such as accuracy, precision, recall and F1-score.

### **4. Dataset – PlantDoc**

PlantDoc is an image dataset designed for plant disease recognition in natural visual conditions. It contains images of plants and disease symptoms captured in realistic environments. Such data is useful because real agricultural images can contain different backgrounds, lighting conditions and levels of image quality.

For this project, PlantDoc is used as the training and evaluation source for the MobileNetV2 image classification model. The exact number of images and classes should be reported from the downloaded version actually used in the experiment, because dataset versions or preprocessing choices can differ.

### **5. Assumptions**

The project assumes that:

- The uploaded image contains a visible crop leaf or plant symptom.
- The image is sufficiently clear for visual analysis.
- The disease belongs to a class represented in the training data.

- The training dataset is representative enough for initial model development.
- The advisory is a decision-support suggestion and not a replacement for an agricultural expert.

## **6. Constraints**

The main constraints are:

- Performance depends on image quality.
- Similar symptoms can occur in different diseases.
- The model may perform poorly on crops or diseases not represented in the dataset.
- Background, lighting, blur and occlusion can affect prediction.
- A deep learning model requires suitable computing resources for training.
- Disease recommendations must be treated carefully because incorrect advice may affect crop health and economic outcomes.

## **7. Application of AI Knowledge**

The project applies artificial intelligence through supervised deep learning and computer vision. The system learns visual patterns such as color changes, spots, texture and leaf damage from labeled training images.

Transfer learning is used so that a pre-trained MobileNetV2 network can be adapted to the crop disease classification task. This reduces the amount of training required compared with building a complete convolutional neural network from the beginning.

## **8. Why MobileNetV2?**

MobileNetV2 is selected because it is a lightweight convolutional neural network designed for efficient image recognition. Its relatively compact architecture makes it suitable for applications where computational resources may be limited.

For a smart agriculture application, a lightweight model can be useful for future deployment on mobile devices, edge devices or low-resource systems. Transfer learning also allows the model to reuse visual features learned from a large image dataset and adapt them to plant disease images.



## **9. Transfer Learning**

In transfer learning, a model trained previously on a large image dataset is reused for a new classification task. The early layers of the network contain general visual features such as edges, shapes and textures. These features can be useful for crop disease recognition.

The MobileNetV2 base model is used as a feature extractor, followed by a classification layer suitable for the PlantDoc classes. During training, the new classification layers are trained first, and selected base layers can later be fine-tuned if required.

## **10. Proposed System Methodology**

The proposed workflow is:

1. Collect the PlantDoc images.
2. Organize images according to their classes.
3. Resize images to the input size required by MobileNetV2.
4. Normalize image pixel values.
5. Apply suitable data augmentation to improve generalization.
6. Load the pre-trained MobileNetV2 model.
7. Replace the original classification head with a new crop disease classifier.
8. Train the model using the training data.
9. Validate the model using unseen validation data.
10. Evaluate the final model using suitable metrics.
11. Accept a new crop image from the user.
12. Predict the disease class and confidence.
13. Display agriculture advisory information.

## **11. Image Preprocessing**

Image preprocessing prepares the input images for the neural network. Images are resized to a consistent resolution and pixel values are normalized according to the model's expected input format.

Data augmentation can include operations such as horizontal flipping, small rotations, zooming and translation. These operations help the model learn useful disease patterns while reducing overfitting to a particular image arrangement.

## **12. MobileNetV2 Architecture**

MobileNetV2 uses depthwise separable convolutions and inverted residual structures to reduce computational cost while maintaining useful image representation capability. The base network extracts visual features from the crop image.

For this project, the original final classification layer is replaced with a task-specific classification head. A global feature representation can be followed by a dropout layer and a dense softmax output layer corresponding to the disease classes.

## **13. Model Training**

The dataset is divided into training and validation/testing portions. The model is trained for a selected number of epochs using an optimizer such as Adam. The learning rate is selected carefully so that the pre-trained features are not changed too aggressively.

Training loss and validation loss are monitored. If validation performance stops improving, early stopping can be used to reduce overfitting.

## **14. Disease Classification**

For a new leaf image, MobileNetV2 generates a probability for each trained disease class. The class with the highest probability is selected as the predicted disease.

The system should display both the predicted class and an appropriate confidence value. If confidence is low, the application can advise the user to capture a clearer image or consult an agricultural expert.

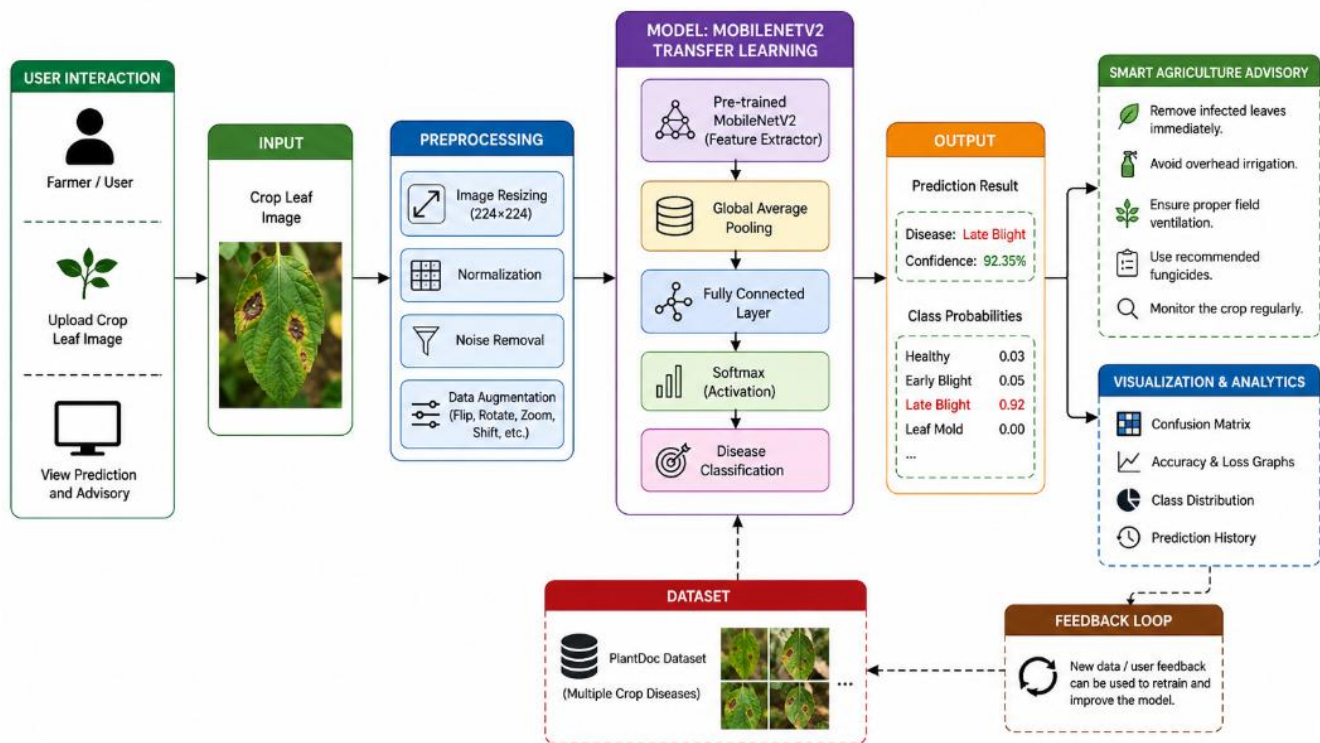
## **15. Smart Agriculture Advisory**

After classification, the system connects the predicted disease to an advisory knowledge base. The advisory may contain:

- Disease name.
- Common visible symptoms.
- Basic prevention or management suggestions.
- Monitoring recommendations.
- General crop-care practices.
- A warning to seek expert advice when chemical treatment or serious crop damage is involved.

The advisory component converts a model prediction into information that is easier for a farmer to understand and act upon.

## 16. System Architecture



## 17. Algorithm

- Step 1: Start the application.
- Step 2: Load the trained MobileNetV2 model.
- Step 3: Load the disease-class mapping and advisory information.
- Step 4: Accept a crop leaf image from the user.
- Step 5: Resize and preprocess the image.
- Step 6: Pass the image through MobileNetV2.
- Step 7: Calculate class probabilities.
- Step 8: Select the class with maximum probability.
- Step 9: Display the predicted disease and confidence.
- Step 10: Retrieve the related agriculture advisory.
- Step 11: Display precautions and recommendations.
- Step 12: End or allow another image to be analyzed.

## 18. Mathematical Model

Let  $x$  represent an input crop image and  $f(x; \theta)$  represent the MobileNetV2 model with parameters  $\theta$ .

The model produces a vector of class scores. A softmax function converts these scores into probabilities:

$$P(y = k | x) = \exp(z_k) / \sum \exp(z_j)$$

where  $z_k$  is the output score for class  $k$ . The predicted class is:

$$\hat{y} = \operatorname{argmax}_k P(y = k | x)$$

The model is trained by minimizing classification loss over the training examples.

## 19. Loss Function

For multi-class disease classification, categorical cross-entropy can be used:

$$L = - \sum y_k \log(p_k)$$

where  $y_k$  is the true class indicator and  $p_k$  is the predicted probability for class  $k$ .

Lower validation loss generally indicates that the model is improving its ability to classify unseen images, although loss should always be considered together with accuracy and other evaluation metrics.

## 20. Tools and Technologies

- Programming Language: Python
- Deep Learning Framework: TensorFlow / Keras
- Model: MobileNetV2 with Transfer Learning
- Dataset: PlantDoc
- Image Processing: Pillow / OpenCV
- Data Handling: NumPy, Pandas
- Evaluation: Scikit-learn
- Visualization: Matplotlib
- Development Environment: VS Code / Jupyter Notebook
- Optional Web Interface: Streamlit

## 21. Hardware Requirements

A basic development system can be used for experimentation. Suggested resources include:

- Modern multi-core processor.
- At least 8 GB RAM.
- Sufficient storage for the dataset and trained model.
- GPU support is recommended for faster deep learning training but is not mandatory for small experiments.

## 22. Software Requirements

- Windows, Linux or macOS.
- Python 3.x.
- TensorFlow/Keras.
- NumPy and Pandas.
- Scikit-learn.
- Matplotlib.
- Pillow or OpenCV.
- VS Code or Jupyter Notebook.
- Streamlit if a web-based user interface is developed.

## 23. User Interface

A simple user interface can contain:

- Project title.
- Image upload option.
- Preview of the uploaded crop image.
- Predict Disease button.
- Predicted disease name.
- Confidence score.
- Symptoms section.
- Smart advisory section.
- General prevention and monitoring suggestions.

The interface should use simple language so that users without technical knowledge can understand the result.

## **24. Model Evaluation**

The trained model should be evaluated using:

- Accuracy – overall proportion of correctly classified images.
- Precision – proportion of predicted positive class samples that are correct.
- Recall – proportion of actual class samples that are correctly identified.
- F1-score – harmonic mean of precision and recall.
- Confusion matrix – shows class-wise correct and incorrect predictions.

Actual numerical results, graphs and confusion matrices must be generated from the final experiment rather than estimated or invented.

## **25. Results and Validation**

The final report should include the actual training and validation accuracy/loss curves, confusion matrix and classification report produced by the trained MobileNetV2 model.

Important observations can include whether the model converges properly, whether training and validation performance are close, which disease classes are difficult to distinguish, and whether data augmentation or fine-tuning improves generalization.

## **26. Testing**

Functional testing can include:

- Upload a valid crop image.
- Verify that the image is displayed correctly.
- Run prediction.
- Verify that a disease class and confidence are displayed.
- Verify that the correct advisory is retrieved.
- Test unclear or unsupported images.
- Test repeated predictions using different crop images.
- Verify that the application does not crash when an invalid file is supplied.

## **27. Comparison with Other AI Models**

MobileNetV2 can be compared with models such as a basic CNN, ResNet50 or EfficientNet. The comparison should consider classification performance, training time, model size and deployment requirements.

A model with slightly higher accuracy may not always be the best engineering choice if it requires substantially more computation. For smart agriculture, a balance between accuracy, speed and deployment cost is important.

## **28. Analysis and Engineering Decision**

MobileNetV2 is selected as the primary model because the project needs a practical image classifier that can potentially be deployed on resource-constrained devices. Transfer learning further reduces training requirements.

The engineering decision should be based on measured experimental results. If another model gives a significant performance improvement with acceptable computational cost, it can be considered for a future version.

## **29. Advantages**

- Automated disease identification.
- Faster preliminary screening.
- Lightweight deep learning architecture.
- Transfer learning reduces training effort.
- Can support early disease awareness.
- Can be extended to mobile or web applications.
- Combines prediction with farmer-friendly advisory information.

## **30. Limitations**

- Prediction quality depends on the training data.
- Some diseases may have visually similar symptoms.
- Real field conditions may differ from the dataset.
- The system cannot guarantee a diagnosis.
- Advisory information must be validated by agricultural experts before being used for high-risk treatment decisions.
- New diseases not represented during training may be misclassified.

### **31. Economic Considerations**

Early detection may help reduce crop losses by encouraging farmers to inspect and manage affected plants sooner. A lightweight model can also reduce the computing cost of future deployment.

However, development, dataset preparation, model training, maintenance and expert validation involve costs. The system should therefore be evaluated based on practical usefulness rather than accuracy alone.

### **32. Environmental Considerations**

The system can contribute indirectly to sustainable agriculture by supporting earlier detection and more targeted crop management. A lightweight model such as MobileNetV2 can also require fewer computational resources than larger deep learning networks.

The application should avoid recommending unnecessary chemical use. Any treatment-related information should be presented responsibly and, where appropriate, accompanied by expert consultation guidance.

### **33. Social Considerations**

The proposed system can make AI-based crop monitoring more accessible to farmers and students. A simple interface and understandable advisory can reduce the technical barrier.

The system should support local languages and consider users with limited internet connectivity in future versions. It should also avoid presenting uncertain predictions as guaranteed facts.

### **34. Safety and Ethical Considerations**

Agricultural recommendations can have financial and environmental consequences. Therefore, the system should clearly communicate uncertainty and avoid claiming that an AI prediction is a confirmed expert diagnosis.

The dataset should be used according to its licensing conditions. Model evaluation should be transparent, and known limitations should be communicated to users. Expert verification is recommended for serious disease outbreaks or chemical treatment decisions.



### **35. Sustainability and SDG Contribution**

The project supports the broader goal of sustainable agriculture by using AI for crop monitoring and early disease awareness. It can be aligned with Sustainable Development Goal 2 (Zero Hunger) and SDG 12 (Responsible Consumption and Production), while also supporting responsible technology use.

The system's sustainability value is strongest when it helps farmers make informed decisions while reducing unnecessary resource use.

### **36. Future Enhancements**

Future improvements can include:

- Larger and more diverse field datasets.
- Additional crop and disease classes.
- Fine-tuning of MobileNetV2.
- Mobile application deployment.
- Offline inference for rural areas.
- Multilingual advisory support.
- Weather and soil information integration.
- Disease severity estimation.
- Explainable AI visualizations such as heatmaps.
- Expert-verified treatment recommendations.
- Continuous model updating with newly collected field images.

### **37. Conclusion**

The AI-Based Crop Disease Detection and Smart Agriculture Advisory System demonstrates how deep learning can be applied to an important agricultural problem. The project uses MobileNetV2 transfer learning with the PlantDoc dataset to classify crop disease images.

The proposed system combines image classification with a simple advisory layer so that the output is more useful than a disease label alone. MobileNetV2 is a practical choice because it provides an efficient architecture suitable for future deployment on devices with limited computing resources.

The final performance should be reported using actual experimental results, including accuracy, precision, recall, F1-score and confusion matrix. With a larger field dataset, expert-validated recommendations and mobile/offline deployment, the system can be developed into a more practical smart agriculture solution.

### **38. Student Reflection**

Through this project, I learned how an artificial intelligence problem can be converted into a practical engineering solution. I understood the importance of selecting an appropriate dataset, preprocessing images, applying transfer learning and evaluating a classification model using multiple metrics.

I also learned that building an AI system is not only about achieving high accuracy. Real-world factors such as computational cost, data quality, safety, ethical use, environmental impact and user understanding are equally important. The project helped me connect deep learning concepts with a real agricultural application.

### **39. References**

1. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.
2. Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S. and Batra, N. (2020). PlantDoc: A Dataset for Visual Plant Disease Detection. Proceedings of the 7th ACM IKDD CoDS and 25th COMAD.
3. TensorFlow/Keras documentation for MobileNetV2 and transfer learning.
4. Scikit-learn documentation for classification metrics and confusion matrices.
5. Relevant PlantDoc dataset documentation and source materials used for the actual experiment.

### **40. Project Summary**

Project Title: AI-Based Crop Disease Detection and Smart Agriculture Advisory System

AI Algorithm: MobileNetV2 Transfer Learning

Dataset: PlantDoc Dataset

Input: Crop/leaf image

Output: Predicted disease class, confidence score and smart agriculture advisory

Main Technologies: Python, TensorFlow/Keras, NumPy, Pandas, Scikit-learn, Matplotlib and optional Streamlit

Main Benefit: Provides an AI-assisted method for preliminary crop disease identification and farmer-friendly advisory support.

**Python Code :**

```
# ALGORITHM: MobileNetV2 Transfer Learning
```

```
# PLATFORM : Windows Python / IDLE
```

```
# =====
```

```
import os
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
from pathlib import Path
```

```
from tkinter import Tk, filedialog, messagebox
```

```
from tensorflow.keras import layers, models
```

```
from tensorflow.keras.applications import MobileNetV2
```

```
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
```

```
from tensorflow.keras.utils import load_img, img_to_array
```

```
from sklearn.metrics import (
```

```
    classification_report,
```

```
    confusion_matrix,
```

ConfusionMatrixDisplay

)

# =====

# 1. BASIC SETTINGS

# =====

IMG\_HEIGHT = 224

IMG\_WIDTH = 224

IMAGE\_SIZE = (IMG\_HEIGHT, IMG\_WIDTH)

BATCH\_SIZE = 32

SEED = 42

IMAGE\_EXTENSIONS = (

".jpg",

".jpeg",

".png",

".bmp",

".webp"

)

print("=" \* 65)

print("PLANT DISEASE DETECTION USING MOBILENETV2")

```
print("=" * 65)

print("TensorFlow Version:", tf.__version__)

print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
# =====
```

```
# 2. SELECT PLANTDOC DATASET FOLDER
```

```
# =====
```

```
# Select the folder that contains the PlantDoc dataset.
```

```
# It can directly contain "train" and "test", or contain another
```

```
# folder which contains them.
```

```
def select_dataset_folder():
```

```
    root = Tk()
```

```
    root.withdraw()
```

```
    root.attributes("-topmost", True)
```

```
    folder = filedialog.askdirectory(
```

```
        title="Select the PlantDoc Dataset Folder"
```

```
    )
```

```
    root.destroy()
```

```
    if not folder:
```

```
raise RuntimeError("No dataset folder was selected.")
```

```
return folder
```

```
EXTRACT_PATH = select_dataset_folder()
```

```
print("\nSelected Dataset Folder:")
```

```
print(EXTRACT_PATH)
```

```
# =====
```

```
# 3. FIND TRAIN AND TEST FOLDERS AUTOMATICALLY
```

```
# =====
```

```
def find_folder(root_path, folder_name):
```

```
    for root, dirs, _ in os.walk(root_path):
```

```
        for directory in dirs:
```

```
            if directory.lower() == folder_name.lower():
```

```
                return os.path.join(root, directory)
```

```
# Also check the selected root itself.
```

```
if os.path.basename(root_path).lower() == folder_name.lower():
```

```
    return root_path
```

```
return None
```

```
TRAIN_DIR = find_folder(EXTRACT_PATH, "train")
```

```
TEST_DIR = find_folder(EXTRACT_PATH, "test")
```

```
print("\nTrain Directory:")
```

```
print(TRAIN_DIR)
```

```
print("\nTest Directory:")
```

```
print(TEST_DIR)
```

```
if TRAIN_DIR is None:
```

```
    raise FileNotFoundError(
```

```
        "TRAIN folder was not found. Select the correct PlantDoc folder."
```

```
    )
```

```
if TEST_DIR is None:
```

```
    raise FileNotFoundError(
```

```
        "TEST folder was not found. Select the correct PlantDoc folder."
```

```
    )
```

```
# =====  
  
# 4. DATASET CLASSES  
  
# =====  
  
classes = sorted([  
    folder  
  
    for folder in os.listdir(TRAIN_DIR)  
  
    if os.path.isdir(os.path.join(TRAIN_DIR, folder))  
  
])  
  
NUM_CLASSES = len(classes)  
  
if NUM_CLASSES == 0:  
    raise ValueError(  
        "No class folders were found inside the TRAIN folder."  
    )  
  
print("\n" + "=" * 65)  
  
print("DATASET INFORMATION")  
  
print("=" * 65)  
  
print("Number of Classes:", NUM_CLASSES)
```



```
print("\nDisease Classes:")
```

```
for i, class_name in enumerate(classes):
```

```
    print(i, ":", class_name)
```

```
# =====
```

```
# 5. COUNT IMAGES
```

```
# =====
```

```
def count_images(folder):
```

```
    total = 0
```

```
    for root, _, files_in_folder in os.walk(folder):
```

```
        for file in files_in_folder:
```

```
            if file.lower().endswith(IMAGE_EXTENSIONS):
```

```
                total += 1
```

```
    return total
```

```
train_image_count = count_images(TRAIN_DIR)
```

```
test_image_count = count_images(TEST_DIR)
```

```
print("\nTraining Images:", train_image_count)
```

```
print("Testing Images :", test_image_count)
```

```
print("Total Images  :", train_image_count + test_image_count)
```

```
# =====
```

```
# 6. CLASS DISTRIBUTION
```

```
# =====
```

```
class_counts = {}
```

```
for class_name in classes:
```

```
    class_path = os.path.join(TRAIN_DIR, class_name)
```

```
    class_counts[class_name] = count_images(class_path)
```

```
print("\nImages per class:")
```

```
for class_name, count in class_counts.items():
```

```
    print(class_name, ":", count)
```

```
# =====
```

```
# 7. CLASS DISTRIBUTION GRAPH
```

```
# =====
```

```
plt.figure(figsize=(15, 6))
```

```
plt.bar(  
    list(class_counts.keys()),  
    list(class_counts.values())  
)  
  
plt.xlabel("Disease Class")  
plt.ylabel("Number of Images")  
plt.title("PlantDoc Training Dataset Class Distribution")  
plt.xticks(rotation=90, fontsize=8)  
plt.tight_layout()  
plt.show()
```

```
# =====
```

```
# 8. LOAD TRAINING DATA
```

```
# =====
```

```
print("\nLoading training dataset...")
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(  
    TRAIN_DIR,  
    validation_split=0.20,  
    subset="training",  
    seed=SEED,
```

```
image_size=IMAGE_SIZE,  
  
batch_size=BATCH_SIZE,  
  
label_mode="int",  
  
class_names=classes  
)
```

```
# =====
```

```
# 9. LOAD VALIDATION DATA
```

```
# =====
```

```
print("Loading validation dataset...")
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(  
  
    TRAIN_DIR,  
  
    validation_split=0.20,  
  
    subset="validation",  
  
    seed=SEED,  
  
    image_size=IMAGE_SIZE,  
  
    batch_size=BATCH_SIZE,  
  
    label_mode="int",  
  
    class_names=classes  
)
```

```
# =====  
  
# 10. LOAD TEST DATA  
  
# =====  
  
print("Loading test dataset...")  
  
test_ds = tf.keras.utils.image_dataset_from_directory(  
    TEST_DIR,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE,  
    shuffle=False,  
    label_mode="int",  
    class_names=classes  
)  
  
class_names = train_ds.class_names  
  
NUM_CLASSES = len(class_names)  
  
print("\nFinal Number of Classes:", NUM_CLASSES)  
  
# =====  
  
# 11. DISPLAY SAMPLE IMAGES  
  
# =====
```

```
plt.figure(figsize=(12, 10))
```

```
for images, labels in train_ds.take(1):
```

```
    number_of_images = min(12, len(images))
```

```
    for i in range(number_of_images):
```

```
        plt.subplot(3, 4, i + 1)
```

```
        plt.imshow(
```

```
            images[i].numpy().astype("uint8")
```

```
        )
```

```
        plt.title(
```

```
            class_names[int(labels[i].numpy())]
```

```
        )
```

```
        plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

## # 12. DATA AUGMENTATION

```
# =====
```

```
data_augmentation = tf.keras.Sequential(  
    [  
        layers.RandomFlip("horizontal"),  
        layers.RandomRotation(0.15),  
        layers.RandomZoom(0.15),  
        layers.RandomTranslation(  
            height_factor=0.1,  
            width_factor=0.1  
        )  
    ],  
    name="data_augmentation"  
)
```

```
print("\nData augmentation created.")
```

```
# =====
```

## # 13. OPTIMIZE DATA PIPELINE

```
# =====
```

```
AUTOTUNE = tf.data.AUTOTUNE
```

```
train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
```

```
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

```
test_ds = test_ds.prefetch(buffer_size=AUTOTUNE)
```

```
# =====
```

```
# 14. LOAD PRETRAINED MOBILENETV2
```

```
# =====
```

```
print("\nLoading MobileNetV2...")
```

```
base_model = MobileNetV2(  
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3),  
    include_top=False,  
    weights="imagenet"  
)
```

```
base_model.trainable = False
```

```
print("MobileNetV2 loaded successfully.")
```

```
print("Pretrained layers frozen.")
```

```
# =====
```



```
# 15. CREATE TRANSFER LEARNING MODEL
```

```
# =====
```

```
inputs = layers.Input(  
    shape=(IMG_HEIGHT, IMG_WIDTH, 3)  
)
```

```
x = data_augmentation(inputs)
```

```
x = preprocess_input(x)
```

```
x = base_model(  
    x,  
    training=False  
)
```

```
x = layers.GlobalAveragePooling2D()(x)
```

```
x = layers.Dropout(0.30)(x)
```

```
x = layers.Dense(  
    128,  
    activation="relu"
```

```
)(x)
```

```
x = layers.Dropout(0.20)(x)
```

```
outputs = layers.Dense(
```

```
    NUM_CLASSES,
```

```
    activation="softmax"
```

```
)(x)
```

```
model = models.Model(
```

```
    inputs=inputs,
```

```
    outputs=outputs
```

```
)
```

```
print("\nMobileNetV2 Transfer Learning Model Created.")
```

```
# =====
```

```
# 16. MODEL SUMMARY
```

```
# =====
```

```
model.summary()
```

```
# =====
```

## # 17. COMPILE MODEL

```
# =====
```

```
model.compile(  
    optimizer=tf.keras.optimizers.Adam(  
        learning_rate=0.001  
    ),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
print("\nModel compiled successfully.")
```

```
# =====
```

## # 18. CALLBACKS

```
# =====
```

```
early_stopping = tf.keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    patience=3,  
    restore_best_weights=True  
)
```

```
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(  
  
    monitor="val_loss",  
  
    factor=0.2,  
  
    patience=2,  
  
    min_lr=0.000001  
  
)
```

```
callbacks = [  
  
    early_stopping,  
  
    reduce_lr  
  
]
```

```
# =====
```

```
# 19. INITIAL TRAINING
```

```
# =====
```

```
print("\n" + "=" * 65)
```

```
print("STARTING INITIAL TRAINING")
```

```
print("=" * 65)
```

```
INITIAL_EPOCHS = 10
```

```
history = model.fit(
```

```
train_ds,  
  
validation_data=val_ds,  
  
epochs=INITIAL_EPOCHS,  
  
callbacks=callbacks  
)
```

```
print("\nInitial training completed.")
```

```
# =====
```

```
# 20. INITIAL ACCURACY GRAPH
```

```
# =====
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(  
  
    history.history["accuracy"],  
  
    label="Training Accuracy"  
)
```

```
plt.plot(  
  
    history.history["val_accuracy"],  
  
    label="Validation Accuracy"  
)
```

```
plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title("MobileNetV2 Training and Validation Accuracy")

plt.legend()

plt.grid(True)

plt.show()
```

```
# =====
```

```
# 21. INITIAL LOSS GRAPH
```

```
# =====
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(

    history.history["loss"],

    label="Training Loss"

)
```

```
plt.plot(

    history.history["val_loss"],

    label="Validation Loss"

)
```

```
plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.title("MobileNetV2 Training and Validation Loss")

plt.legend()

plt.grid(True)

plt.show()
```

```
# =====
```

```
# 22. FINE-TUNING MOBILENETV2
```

```
# =====
```

```
print("\n" + "=" * 65)

print("STARTING FINE-TUNING")

print("=" * 65)
```

```
base_model.trainable = True
```

```
# Freeze all but the last 30 MobileNetV2 layers.
```

```
for layer in base_model.layers[:-30]:
```

```
    layer.trainable = False
```

```
# Keep BatchNormalization layers frozen for stable fine-tuning.
```

```

for layer in base_model.layers:

    if isinstance(layer, layers.BatchNormalization):

        layer.trainable = False


trainable_layers = sum(

    1

    for layer in base_model.layers

    if layer.trainable

)


print("Trainable MobileNetV2 layers:", trainable_layers)


# =====

# 23. RECOMPILE FOR FINE-TUNING

# =====


model.compile(

    optimizer=tf.keras.optimizers.Adam(

        learning_rate=0.00001

    ),

    loss="sparse_categorical_crossentropy",

    metrics=["accuracy"]

)

```



```
print("Model recompiled for fine-tuning.")
```

```
fine_callbacks = [  
    tf.keras.callbacks.EarlyStopping(  
        monitor="val_loss",  
        patience=3,  
        restore_best_weights=True  
    ),  
    tf.keras.callbacks.ReduceLROnPlateau(  
        monitor="val_loss",  
        factor=0.2,  
        patience=2,  
        min_lr=0.0000001  
    )  
]
```

```
# =====
```

```
# 24. FINE-TUNE MODEL
```

```
# =====
```

```
FINE_TUNE_EPOCHS = 10
```

```
history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=FINE_TUNE_EPOCHS,
    callbacks=fine_callbacks
)
```

```
print("\nFine-tuning completed.")
```

```
# =====
```

```
# 25. FINAL TEST EVALUATION
```

```
# =====
```

```
print("\n" + "=" * 65)
```

```
print("FINAL MODEL EVALUATION")
```

```
print("=" * 65)
```

```
test_loss, test_accuracy = model.evaluate(
    test_ds,
    verbose=1
)
```

```
print("\nTest Loss:", round(test_loss, 4))
```

```
print(  
    "Test Accuracy:",  
    round(test_accuracy * 100, 2),  
    "%"  
)
```

```
# =====
```

```
# 26. GENERATE TEST PREDICTIONS
```

```
# =====
```

```
y_true = []
```

```
y_pred = []
```

```
for images, labels in test_ds:
```

```
    predictions = model.predict(  
        images,  
        verbose=0  
    )
```

```
    predicted_classes = np.argmax(  
        predictions,  
        axis=1  
    )
```

```

y_true.extend(labels.numpy())

y_pred.extend(predicted_classes)


y_true = np.array(y_true)

y_pred = np.array(y_pred)


print("\nPredictions generated.")


# =====

# 27. CLASSIFICATION REPORT

# =====


print("\n" + "=" * 65)

print("CLASSIFICATION REPORT")

print("=" * 65)


report = classification_report(

    y_true,

    y_pred,

    labels=np.arange(NUM_CLASSES),

    target_names=class_names,

    zero_division=0

```

```
)
```

```
print(report)
```

```
# =====
```

```
# 28. CONFUSION MATRIX
```

```
# =====
```

```
cm = confusion_matrix(
```

```
    y_true,
```

```
    y_pred,
```

```
    labels=np.arange(NUM_CLASSES)
```

```
)
```

```
fig, ax = plt.subplots(figsize=(14, 12))
```

```
disp = ConfusionMatrixDisplay(
```

```
    confusion_matrix=cm,
```

```
    display_labels=class_names
```

```
)
```

```
disp.plot(
```

```
    ax=ax,
```

```
xticks_rotation=90,  
  
colorbar=False  
  
)
```

```
plt.title("PlantDoc - MobileNetV2 Confusion Matrix")  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
  
# 29. SELECT NEW IMAGE  
  
# =====
```

```
print("\n" + "=" * 65)  
  
print("NEW PLANT LEAF PREDICTION")  
  
print("=" * 65)
```

```
print("Select a plant leaf image in the file dialog.")
```

```
root = Tk()  
  
root.withdraw()  
  
root.attributes("-topmost", True)
```

```
IMAGE_FILE = filedialog.askopenfilename(
```

```
title="Select Plant Leaf Image",

filetypes=[

    ("Image files", "*.jpg *.jpeg *.png *.bmp *.webp"),

    ("All files", "*.*")

]

)
```

```
root.destroy()
```

```
if not IMAGE_FILE:
```

```
    print("No image selected.")
```

```
else:
```

```
# =====
```

```
# 30. LOAD AND PREDICT IMAGE
```

```
# =====
```

```
img = load_img(
```

```
    IMAGE_FILE,
```

```
    target_size=(IMG_HEIGHT, IMG_WIDTH)
```

```
)
```

```
img_array = img_to_array(img)
```

```
# Use the SAME preprocessing used during model training.
```

```
img_array = preprocess_input(img_array)
```

```
img_array = np.expand_dims(
```

```
    img_array,
```

```
    axis=0
```

```
)
```

```
prediction = model.predict(
```

```
    img_array,
```

```
    verbose=0
```

```
)
```

```
predicted_index = int(
```

```
    np.argmax(prediction[0])
```

```
)
```

```
predicted_class = class_names[predicted_index]
```

```
confidence = (
```

```
    float(prediction[0][predicted_index]) * 100
```

```
)
```



```
# =====
```

```
# 31. DISPLAY PREDICTION
```

```
# =====
```

```
plt.figure(figsize=(6, 6))
```

```
plt.imshow(img)
```

```
plt.title(
```

```
    f'{predicted_class}\n'
```

```
    f'Confidence: {confidence:.2f}%'
```

```
)
```

```
plt.axis("off")
```

```
plt.show()
```

```
print("\n" + "=" * 65)
```

```
print("PREDICTION RESULT")
```

```
print("=" * 65)
```

```
print(
```

```
    "Predicted Disease:",
```

```
    predicted_class
)
```

```
print(
    "Confidence:",
    f"{confidence:.2f}%"
)
```

```
# =====
# 32. SMART AGRICULTURE ADVISORY
# =====
```

```
def get_advisory(disease):
```

```
    disease_lower = disease.lower()
```

```
    if "healthy" in disease_lower:
```

```
        return ""
```

The plant appears HEALTHY.

Recommended Actions:

- Continue regular crop monitoring.
- Maintain proper irrigation.

- Maintain suitable soil nutrition.
- Monitor the plant for early disease symptoms.
- Maintain good field sanitation.

"""

```
elif (
    "early_blight" in disease_lower
    or "early blight" in disease_lower
):
    return """
```

EARLY BLIGHT DETECTED.

Recommended Actions:

- Remove severely affected leaves.
- Maintain good field sanitation.
- Improve air circulation around plants.
- Avoid prolonged leaf wetness.
- Monitor nearby plants for symptoms.
- Follow appropriate agricultural disease-management guidance.

"""

```
elif (
    "late_blight" in disease_lower
```

or "late blight" in disease\_lower

):

return ""

LATE BLIGHT DETECTED.

Recommended Actions:

- Remove severely infected plant material.
- Reduce prolonged leaf wetness.
- Improve air circulation.
- Monitor surrounding plants carefully.
- Maintain field sanitation.
- Seek appropriate agricultural guidance before treatment.

""

elif (

"leaf\_mold" in disease\_lower

or "leaf mold" in disease\_lower

):

return ""

LEAF MOLD DETECTED.

Recommended Actions:

- Improve ventilation around plants.

- Avoid excessive moisture.
- Avoid unnecessary overhead irrigation.
- Remove heavily affected leaves.
- Monitor disease progression.

"""

elif "rust" in disease\_lower:

return ""

RUST DISEASE DETECTED.

Recommended Actions:

- Remove affected plant material.
- Maintain field sanitation.
- Improve air circulation.
- Regularly inspect new leaves.
- Monitor disease progression.

"""

elif (

"bacterial\_spot" in disease\_lower

or "bacterial spot" in disease\_lower

):

return ""

BACTERIAL SPOT DETECTED.

Recommended Actions:

- Remove severely affected leaves.
- Maintain good crop sanitation.
- Avoid unnecessary overhead watering.
- Monitor nearby plants.
- Seek appropriate agricultural guidance.

"""

else:

return """

DISEASE DETECTED.

Recommended Actions:

- Monitor the affected crop regularly.
- Remove severely affected plant material where appropriate.
- Maintain good crop sanitation.
- Avoid unnecessary moisture on foliage.
- Consult a qualified agricultural expert before applying treatments.

"""

# =====

```
# 33. DISPLAY AGRICULTURAL ADVISORY
```

```
# =====
```

```
print("\n" + "=" * 65)
```

```
print("SMART AGRICULTURE ADVISORY")
```

```
print("=" * 65)
```

```
print(
```

```
    get_advisory(predicted_class)
```

```
)
```

```
# =====
```

```
# 34. SHOW TOP 5 PREDICTIONS
```

```
# =====
```

```
print("\n" + "=" * 65)
```

```
print("TOP 5 PREDICTIONS")
```

```
print("=" * 65)
```

```
top_k = min(5, NUM_CLASSES)
```

```
top_indices = np.argsort(
```

```
    prediction[0]
```

```
)[-top_k:][::-1]
```

```
for rank, index in enumerate(
```

```
    top_indices,
```

```
    start=1
```

```
):
```

```
    print(
```

```
        rank,
```

```
        ".",
```

```
        class_names[index],
```

```
        "->",
```

```
        f'{prediction[0][index] * 100:.2f} %'
```

```
    )
```

```
# =====
```

```
# 35. SAVE TRAINED MODEL
```

```
# =====
```

```
# The model will be saved in the same folder as this Python file.
```

```
SCRIPT_FOLDER = os.path.dirname(
```

```
    os.path.abspath(__file__)
```

```
)
```



```
MODEL_PATH = os.path.join(  
    SCRIPT_FOLDER,  
    "PlantDoc_MobileNetV2.keras"  
)
```

```
model.save(MODEL_PATH)
```

```
print("\n" + "=" * 65)
```

```
print("MODEL SAVED")
```

```
print("=" * 65)
```

```
print("Model location:")
```

```
print(MODEL_PATH)
```

```
print("\nProject implementation completed successfully!")
```

```
# =====
```

```
# END
```

```
# =====
```

## Output :

```
=====
DATASET INFORMATION
=====
Number of Classes: 28

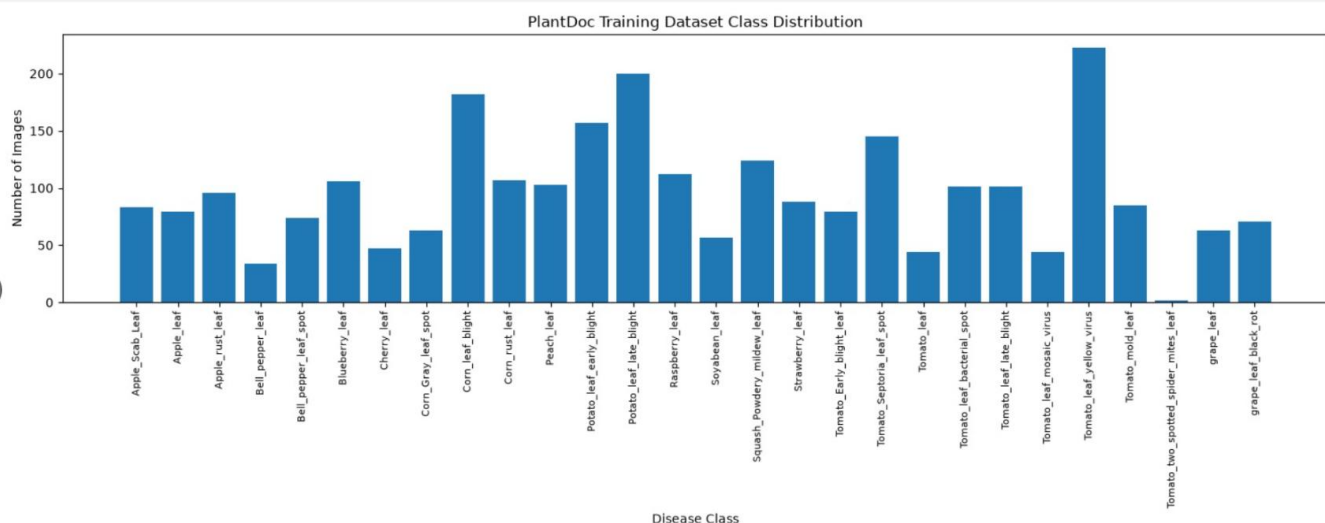
Disease Classes:
0 : Apple_Scab_Leaf
1 : Apple_leaf
2 : Apple_rust_leaf
3 : Bell_pepper_leaf
4 : Bell_pepper_leaf_spot
5 : Blueberry_leaf
6 : Cherry_leaf
7 : Corn_Gray_leaf_spot
8 : Corn_leaf_blight
9 : Corn_rust_leaf
10 : Peach_leaf
11 : Potato_leaf_early_blight
12 : Potato_leaf_late_blight
13 : Raspberry_leaf
14 : Soyabean_leaf
15 : Squash_Powdery_mildew_leaf
16 : Strawberry_leaf
17 : Tomato_Early_blight_leaf
18 : Tomato_Septoria_leaf_spot
19 : Tomato_leaf
20 : Tomato_leaf_bacterial_spot
21 : Tomato_leaf_late_blight
22 : Tomato_leaf_mosaic_virus
23 : Tomato_leaf_yellow_virus
24 : Tomato_mold_leaf
25 : Tomato_two_spotted_spider_mites_leaf
26 : grape_leaf
27 : grape_leaf_black_rot

Training Images: 2670
Testing Images : 252
Total Images   : 2922
```

```

Images per class:
Apple_Scab_Leaf : 83
Apple_leaf : 79
Apple_rust_leaf : 96
Bell_pepper_leaf : 34
Bell_pepper_leaf_spot : 74
Blueberry_leaf : 106
Cherry_leaf : 47
Corn_Gray_leaf_spot : 63
Corn_leaf_blight : 182
Corn_rust_leaf : 107
Peach_leaf : 103
Potato_leaf_early_blight : 157
Potato_leaf_late_blight : 200
Raspberry_leaf : 112
Soyabean_leaf : 57
Squash_Powdery_mildew_leaf : 124
Strawberry_leaf : 88
Tomato_Early_blight_leaf : 79
Tomato_Septoria_leaf_spot : 145
Tomato_leaf : 44
Tomato_leaf_bacterial_spot : 101
Tomato_leaf_late_blight : 101
Tomato_leaf_mosaic_virus : 44
Tomato_leaf_yellow_virus : 223
Tomato_mold_leaf : 85
Tomato_two_spotted_spider_mites_leaf : 2
grape_leaf : 63
grape_leaf_black_rot : 71

```



## F. Common Assessment Rubric

| Assessment Criterion                                                   | Marks      |
|------------------------------------------------------------------------|------------|
| Problem understanding & formulation                                    | 10         |
| Application of course/domain knowledge                                 | 20         |
| Solution methodology / design / implementation                         | 20         |
| Use of appropriate modern tools / techniques                           | 10         |
| Results, testing & validation                                          | 15         |
| Analysis, trade-offs & justification                                   | 15         |
| Broader considerations / professional responsibility, where applicable | 5          |
| Technical documentation & reflection                                   | 5          |
| <b>TOTAL</b>                                                           | <b>100</b> |

## G. CO–PO–Assessment Mapping

| Assessment Component               | CO  | PO(s) | Bloom's Level | Marks |
|------------------------------------|-----|-------|---------------|-------|
| Problem formulation                | 1,2 | 2     | L3/L4         | 10    |
| Application of knowledge           | 3,4 | 1,2   | L3/L4         | 20    |
| Solution / Design / Implementation | 4,5 | 3,5   | L4/L5/L6      | 20    |
| Modern tool usage                  | 4,5 | 5     | L3/L4         | 10    |

|                            |           |       |       |    |
|----------------------------|-----------|-------|-------|----|
| Validation                 | 4         | 4     | L4/L5 | 15 |
| Analysis & justification   | 3,4       | 2,4   | L4/L5 | 15 |
| Broader considerations     | 5         | 6,7,8 | L4/L5 | 5  |
| Documentation & reflection | 3,4,<br>5 | 10    | L3/L4 | 5  |

By:

MITHRA DEVI S

192524203

BTech AIDS